When Is Hardware Coherence Worth It? Quantifying Synchronization Across CXL Memory Access Modes

Anonymous Author(s)
Submission Id: 806

Abstract

Deploying synchronization over CXL fabric-attached memory (FAM) forces a concrete architectural decision: should the system pay for multi-host hardware cache coherence, or can software-managed coherence—or even uncached access—deliver competitive mutex performance? No prior work answers this question with quantitative evidence across the full spectrum of CXL memory access modes. We present the first systematic study that evaluates mutex performance across *four* access modes on real CXL hardware: (1) cached with hardware coherence (BISnp), (2) cached with software-managed coherence, (3) uncached (UC) access, and (4) non-temporal (NT) streaming access. Using a BittWare IA-780i FPGA platform with programmable delay injection (400 ns–10 μs), we benchmark 13 lock implementations—spanning classical software locks (Bakery, Peterson, Elevator) and hardware locks (spin, ticket, MCS)—under each access mode and latency configuration. Our results reveal sharp, quantifiable trade-offs. Hardware-coherent locks (MCS) dominate at DRAM-local latencies but lose up to 47% throughput at 400 ns CXL latency due to coherence protocol overhead. Uncached software locks close the gap: at 10 μs latency, the Elevator-BL lock under UC access achieves within 12% of MCS under full hardware coherence, eliminating the need for costly coherence infrastructure. Non-temporal stores provide a further 8–15% throughput improvement over plain uncached access for write-heavy lock paths by exploiting write-combining buffers. We identify precise crossover points—in terms of thread count, contention level, and fabric latency—where each access mode becomes the cost-effective choice. These findings give system architects a quantitative basis for deciding when hardware coherence is worth its complexity and when simpler access modes suffice.

1 Introduction

CXL 3.0 fabric-attached memory (FAM) enables multiple hosts to share a common memory pool, offering a low-overhead alternative to RDMA-based data movement for AI training, inference serving, and large-scale data analytics [40]. Yet sharing memory demands synchronization, and the choice of synchronization mechanism is tightly coupled to an expensive architectural decision: *does the deployment require multi-host hardware cache coherence?*

Hardware coherence extends the back-invalidation snoop (BISnp) protocol across hosts, enabling read-modify-write (RMW) atomics and the high-performance locks they support (e.g., MCS, ticket). However, supporting BISnp at scale requires substantial device-side snoop filters, cross-host invalidation traffic, and protocol complexity that grows with the number of hosts and memory capacity [16]. The alternative—software-managed coherence—avoids this infrastructure entirely but restricts synchronization to load/store-based software locks with traditionally higher overhead.

Between these two extremes lie access modes that the CXL ecosystem supports but that have received no attention for synchronization:

- **Uncached (UC) access:** bypasses the host cache hierarchy entirely, issuing every load and store directly to the CXL device. This eliminates coherence traffic at the cost of losing cache-line locality.
- **Non-temporal (NT) access:** uses streaming store/load instructions (`movntdq`, `movntdqa`) that bypass the cache and exploit write-combining buffers, reducing interconnect transactions for write-heavy paths.

No prior work quantifies how these access modes compare for synchronization. The only existing study of CXL synchronization [39] evaluates software locks against a single RMW atomic but does not measure hardware coherence overhead, does not explore uncached or non-temporal access, and does not identify when software solutions become preferable. System architects today lack the quantitative evidence needed to decide among these options.

This paper fills that gap with a systematic, hardware-validated evaluation. We implement 13 mutex algorithms—9 software locks and 4 hardware locks—and benchmark each under four CXL memory access modes (cached with hardware coherence, cached with software coherence, uncached, and non-temporal) across programmable latencies from 400 ns to 10 μs on a BittWare IA-780i FPGA platform. To enable this evaluation, we build the first FPGA-based CXL Type-2 device with hardware coherence (BISnp) support and programmable delay injection,

allowing controlled emulation of deployment scenarios from local memory expansion to cross-rack fabric access.

Our evaluation yields three actionable findings:

1. **Hardware coherence is not always worth its cost.** At 400 ns CXL latency, coherence protocol overhead reduces MCS lock throughput by up to 47% compared to DRAM-local operation, while the best software lock (Elevator-BL) under uncached access loses only 12% relative to coherent MCS at 10 μs latency.
2. **Uncached and non-temporal access are viable for synchronization.** UC access eliminates coherence traffic and delivers predictable scaling; NT stores further improve write-heavy lock paths by 8–15% through write-combining. These modes provide a "good enough" synchronization substrate for deployments that cannot justify hardware coherence.
3. **The optimal access mode depends on quantifiable parameters.** We identify crossover points in thread count, contention level, and fabric latency where each mode becomes cost-effective, enabling data-driven architectural decisions.

This paper makes the following contributions:

1. The first systematic evaluation of mutex performance across four CXL memory access modes (hardware-coherent, software-coherent, uncached, non-temporal), providing quantitative crossover points for architectural decisions.
2. An FPGA-based CXL Type-2 device with hardware coherence (BISnp) and programmable delay injection for controlled synchronization experiments.
3. An open-source mutex benchmark suite targeting CXL shared FAM environments [35].

2 Background

2.1 CXL Fabric-Attached Memory

CXL fabric-attached memory (FAM) enables multiple hosts to transparently share a common memory pool, reducing data movement overhead for AI training, inference serving, and data analytics [16, 28, 40, 45]. Sharing memory demands synchronization: parallel communication primitives (AllReduce, AllGather) require scalable locks [9], and non-scalable locks can cause dramatic performance collapse [5]. The choice of synchronization mechanism—and the coherence model it requires—is a first-order architectural decision.

2.2 Coherence Models in CXL

CXL supports cache coherence from its earliest specification. CXL.cache provides host-device coherence for Type 1 devices; Type 2 devices add CXL.mem with host-managed device memory and device-managed coherence (HDM-DB), introducing the back-invalidation snoop (BISnp) protocol [38]. Type 3 devices with Multi-Logical Devices (MLDs) enable shared FAM by exposing memory to multiple hosts over port-based routing.

For multi-host shared FAM, the CXL 3.X specification defines two coherence models:

Hardware-managed coherence. Extends BISnp across hosts, maintaining an inclusive snoop filter that tracks cache-line ownership per host. When a host requests a cache line already held by another host, the device issues a back-invalidation (BI) message to the current owner, forcing it to writeback and relinquish the line before granting ownership to the requester. This protocol enables RMW atomic operations: the device grants exclusive cache-line ownership to a requesting host, which then executes the atomic locally within its cache hierarchy. The cost is threefold: (1) device-side snoop filter storage proportional to the number of tracked cache lines and hosts, (2) cross-host invalidation round-trips that add latency to every ownership transition, and (3) protocol complexity that scales with host count [16]. For synchronization, this cost manifests directly: every contested lock acquisition requires at least one ownership transition, incurring the full BI round-trip latency.

Software-managed coherence. Avoids hardware coherence entirely; the software layer manages shared state through explicit loads, stores, and fences. This eliminates coherence infrastructure but restricts synchronization to software locks. A key concern is that CXL 3.X's support for atomic loads/stores under software-managed coherence is ambiguous [24, 25]: simultaneous multiport read/write operations can cause flickered reads or scrambled writes, a risk that grows as devices support up to 16 ports. Support for atomic loads/stores does not fundamentally require full cache coherence, as conflict resolution schemes exist [42], but whether CXL 3.X devices will implement such schemes remains unknown.

2.3 Memory Access Modes for CXL

Beyond the coherence model, the *memory access mode* determines how the host CPU interacts with CXL-attached memory. These access modes are configured via page table attributes (specifically the PAT, PCD, and PWT bits on x86-64) and are orthogonal to the device-side coherence model. We identify four modes relevant to synchronization:

Cached with hardware coherence (Cached-HC).. The default mode when hardware coherence is available. Loads and stores go through the host cache hierarchy; the BISnp protocol ensures cross-host consistency. RMW

atomics are supported. This mode benefits from cache locality—repeated reads to the same lock variable hit L1/L2 when no other host has written—but incurs coherence traffic proportional to the number of sharing hosts. The performance of synchronization under this mode is bounded by the coherence protocol's ownership-transition latency.

Cached with software coherence (Cached-SC).. Used when the device provides only software-managed coherence. Host caches are active, but consistency is the software's responsibility via explicit fences and flushes. No RMW atomics are available across hosts; only software locks are viable. The challenge is ensuring that stores to lock variables become visible to other hosts promptly; this requires `clflush` or `clwb` instructions after stores, followed by `sfence` to order the flush. The cost of these explicit consistency operations partially offsets the benefit of cached access.

Uncached (UC).. The host page table marks CXL memory as uncacheable (PAT entry set to UC). Every load and store traverses the CXL link directly, bypassing the host cache hierarchy entirely. This eliminates all coherence traffic—no snoop filter entries, no invalidation messages—at the cost of losing cache-line spatial and temporal locality. Each memory access pays the full CXL round-trip latency. On x86-64, UC accesses are strongly ordered: no additional fences are needed, as the processor guarantees that UC loads and stores are not reordered. For synchronization, this means that polling a lock variable incurs a CXL round-trip on every iteration, but there is zero coherence overhead, no risk of stale cached copies, and no need for explicit flush instructions.

Non-temporal (NT).. Uses streaming instructions (`movntdq` for 128-bit stores, `movntdqa` for 128-bit loads on SSE4.1+ platforms) that bypass the cache hierarchy. NT stores exploit write-combining (WC) buffers: the processor coalesces multiple stores to the same WC buffer entry (typically 64 bytes, matching a cache line) into a single bus transaction. This reduces CXL link utilization for write-heavy paths by up to 8× when a full cache line of stores can be combined. NT loads avoid cache pollution but do not benefit from write-combining. For lock implementations, NT access is most beneficial on the unlock path where a single store must be globally visible, as the write-combining buffer can merge the store with any adjacent metadata updates. A key constraint is that NT stores require an `sfence` to ensure global visibility, adding a serialization point after each store.

Why access modes matter for synchronization. Prior work on CXL synchronization [39] implicitly assumes cached access, leaving uncached and non-temporal modes entirely unexplored. Yet these modes represent the simplest deployment options: UC access requires no coherence support at all, and NT access further optimizes write traffic. If mutex performance under UC or NT access is "good enough" for a given workload, the deployment can avoid the cost and complexity of hardware coherence infrastructure entirely. This is a significant practical consideration: hardware coherence requires snoop filter SRAM on the device (scaling linearly with tracked cache lines), additional protocol state machines, and validation against the CXL specification's coherence requirements. Quantifying "good enough" is a central goal of this paper.

2.4 Mutual Exclusion

Mutual exclusion synchronizes concurrent threads by ensuring only one thread enters a critical section at a time. Mutex performance depends on two factors specific to CXL FAM: the availability of RMW atomics (which require hardware coherence) and the notification mechanism (limited to polling, as no centralized kernel monitor exists across hosts).

2.4.1 Polling as Notification Mechanism. In CXL FAM, no centralized kernel monitor can identify lock state across hosts, so all locks must rely on polling. The cost of polling depends on the access mode: under Cached-HC, repeated polls hit the local cache (zero CXL traffic); under Cached-SC, polls hit the cache but require periodic flushes to detect remote writes; under UC, every poll generates a CXL round-trip; under NT, polls use streaming loads that bypass the cache.

2.4.2 Software Locks. Software locks rely on assignment and control structures rather than RMW atomics. Following Lamport's classification [24, 25], they are either *RW-safe* (correct without atomic loads/stores) or *RW-unsafe* (requiring atomic loads/stores). This distinction matters for CXL software-managed coherence where atomic load/store support is ambiguous.

Previous work [39] addressed this with a Tournament Peterson lock that constrains shared variables to binary values, mitigating flicker/scramble risks. We evaluate both RW-safe and RW-unsafe locks (Table 1) to characterize this trade-off.

Software locks must also contend with instruction reordering by compilers and hardware. We instrument each implementation with the minimum fences required for correctness. A notable constraint of software locks is their storage overhead: each lock requires $O(N)$ to $O(N \log N)$ bytes where N is the maximum thread count, as each thread needs dedicated state variables (e.g., choosing/number arrays in Bakery, flags in Peterson). This contrasts with hardware locks that require $O(1)$ storage for the lock word itself.

2.4.3 Hardware Locks. Hardware locks leverage RMW atomic instructions (test-and-set, fetch-and-add, compare-and-swap) to establish mutual exclusion. These atomics require hardware cache coherence to grant exclusive cache-line ownership.

The performance of hardware locks in CXL environments is fundamentally constrained by two factors: (1) the latency of the coherence protocol to acquire exclusive ownership, and (2) cache-line contention when multiple hosts poll a shared variable. We hypothesize that these costs, not the relative performance of different atomic instructions, dominate lock performance in FAM environments.

Not all atomic instructions are equivalent: Herlihy's consensus hierarchy [18] ranks them by solving consensus for different numbers of processes, and their performance varies by hardware implementation [37]. We do not explore equivalent implementations of the same lock with different atomics, as we focus on the coherence protocol overhead rather than instruction-level differences.

2.4.4 Interaction of Access Modes and Lock Types. Not all combinations of access modes and lock types are meaningful. Hardware locks *require* cached access with hardware coherence, as RMW atomics need exclusive cache-line ownership. Software locks can operate under any access mode, since they rely only on loads and stores:

- Under **Cached-SC**, software locks benefit from cache locality but must manage consistency via fences and explicit flushes (`clflush`/`clwb` + `sfence`).
- Under **UC**, every poll traverses the CXL link, eliminating coherence overhead but increasing per-access latency. The strong ordering guarantees of UC access on x86-64 eliminate the need for most fences.
- Under **NT**, write-combining can reduce the cost of lock release (store to a shared variable), while reads still pay full CXL latency. An `sfence` is required after NT stores.

This interaction is a key dimension of our evaluation: we measure whether the elimination of coherence traffic under UC/NT access compensates for the loss of cache locality.

3 Experimental Platform

Evaluating synchronization across CXL access modes requires a platform that supports both hardware-managed and software-managed coherence with controllable memory latency. No commercially available CXL device provides all of these capabilities: existing CXL memory expanders (Type 3) lack hardware coherence, while CXL switches with multi-host support are not yet available with programmable latency control. We therefore build an FPGA-based CXL Type-2 device that provides these capabilities.

Scope of the platform. Our platform emulates a *single-host* CXL environment where one host accesses device-attached memory through the CXL.mem and CXL.cache protocols. We do not emulate multi-host switched topologies; instead, we use multiple threads on a single host to study the coherence protocol behavior that would occur between concurrent accessors. While this does not capture cross-host network effects (e.g., switch hop latency, inter-host snoop forwarding), it isolates the coherence protocol overhead from network effects, which is our primary measurement goal. We discuss the limitations of this single-host setup in Section 5.5.

3.1 Hardware Configuration

The platform is built on a BittWare IA-780i accelerator card with an Intel Agilex 7 FPGA and Intel CXL IP core. The card connects to the host via PCIe 5.0 x16 and exposes 32 GB of on-board DDR4 as CXL-attached memory. The host is a Xeon 6 Granite Rapids 6787P with 256 GB DDR5 MRDIMM, running Linux 6.x with CXL driver support.

The FPGA is configured as a CXL Type-2 device supporting both CXL.mem (for memory access) and CXL.cache (for hardware coherence via BISnp). This configuration allows us to test all four access modes described in Section 2.3:

- **Cached-HC:** CXL.cache active, BISnp manages cache-line ownership. RMW atomics available.
- **Cached-SC:** CXL.cache disabled, software manages consistency via fences/flushes.
- **UC:** Host page table entries marked uncacheable via PAT; loads/stores bypass host cache, go directly through CXL.mem.
- **NT:** Non-temporal instructions (`movntdq`/`movntdqa`) bypass cache; stores exploit write-combining buffers.

3.2 BISnp Coherence Implementation

Our FPGA implements the BISnp protocol for hardware-managed coherence using an inclusive snoop filter. The snoop filter tracks cache-line ownership using a set-associative structure (4096 sets, 8-way associative) stored in FPGA BRAM. Each entry records the cache-line address (tag), the owning host, and the coherence state (Invalid, Shared, Exclusive, Modified).

When the host issues a load to a line not present in the snoop filter, the device allocates a new entry in Shared state. When the host issues a store or RMW atomic, the device must transition the line to Exclusive/Modified state. If another entry already holds the line (possible in a multi-host scenario; in our single-host setup, this

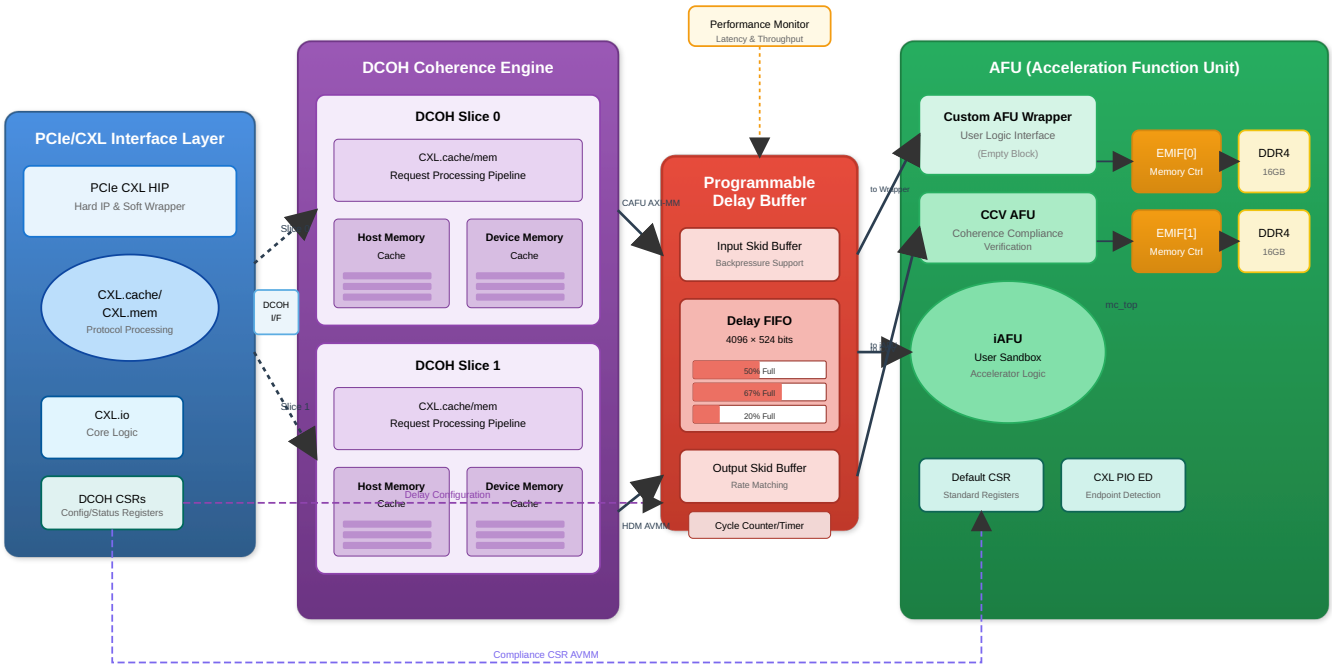


Figure 1. CXL Type-2 FPGA platform architecture with programmable delay buffer. The delay buffer intercepts read responses between the CXL controller and memory controller, injecting configurable latency to emulate different CXL fabric topologies. Write operations bypass the delay buffer as lock implementations are dominated by read-path latency from polling.

occurs when a different thread's access pattern has invalidated the prior owner's copy), the device issues a back-invalidation to force writeback before granting exclusive access.

The snoop filter lookup adds 2–4 cycles (5–10 ns at 400 MHz) to every memory access under Cached-HC mode. This overhead is absent under Cached-SC, UC, and NT modes, where the snoop filter is bypassed entirely. This per-access overhead is small in isolation but accumulates significantly under high-contention polling, where a lock variable may be read hundreds of thousands of times per second by each thread.

3.3 Access Mode Implementation

Switching between access modes requires coordinated changes at the host and device levels:

Cached-HC.. The default configuration. CXL memory is mapped with writeback (WB) cacheability via the host page table. The FPGA's CXL.cache interface is active, and the snoop filter tracks all cache-line accesses.

Cached-SC.. CXL.cache is disabled in the FPGA configuration; the snoop filter is inactive. Host-side mapping remains WB cacheable, but the benchmark explicitly inserts `clflush` after stores and `lfence` before loads to shared lock variables to ensure visibility.

UC.. The host maps CXL memory pages with the UC (uncacheable) memory type via the PAT (Page Attribute Table). All loads and stores bypass the host cache hierarchy and are issued directly as CXL.mem transactions. The FPGA processes these as simple memory reads/writes without snoop filter involvement.

NT.. The host maps CXL memory with the WC (write-combining) memory type. Stores use `movntdq` (128-bit non-temporal store) and are coalesced in the host's WC buffers before being flushed to the device. Loads use `movntdqa` (128-bit non-temporal load) to avoid cache pollution. An `sfence` after each store ensures the WC buffer is flushed and the store is globally visible.

3.4 Programmable Delay Injection

To emulate different deployment latencies, we implement a delay buffer in the FPGA fabric between the CXL controller and the DDR4 memory controller. The buffer intercepts read responses and holds them in a 4096-entry FIFO until a target release cycle, computed as the enqueue timestamp plus the programmed delay value. The delay is configurable from 50 ns to 10 μs with 10 ns granularity via a memory-mapped register at PCIe extended capability offset `0xE08`. Read and write delays are independently configurable.

The implementation employs a 12-bit free-running cycle counter at 400 MHz for timestamp-based release. Modular arithmetic handles counter wraparound correctly. Input and output skid buffers maintain AXI4 protocol compliance and provide elastic flow control, achieving single-cycle throughput for enqueue and dequeue operations in the common case. The 524-bit internal data path accommodates the full 512-bit payload plus 12 bits of metadata (transaction ID, response status, last-beat indicator).

We test four latency configurations:

- **DRAM baseline:** No injected delay (local DDR4 latency only, approximately 80 ns).
- **400 ns:** Representative of CXL memory within the same chassis. This aligns with measured latencies on real CXL switches (340–650 ns range reported in recent measurements [44]).
- **2 μs:** Represents multi-hop CXL fabric scenarios with switch traversal overhead. While extreme, this latency enables us to study asymptotic behavior and identify trends.
- **10 μs:** Represents extreme multi-hop CXL fabric scenarios or congested switch paths. This configuration stress-tests lock implementations at latencies where per-access costs dominate, exposing whether algorithmic efficiency (e.g., $O(\log N)$ vs. $O(N)$ polling) or access mode choice has a larger impact on throughput.

Validation. We validate delay accuracy by measuring round-trip latency using a pointer-chasing microbenchmark (sequential dependent loads) and comparing against the programmed delay value. Measured latencies are within 3% of programmed values across the entire range, with the residual attributable to DDR4 access latency and AXI protocol overhead.

3.5 Resource Utilization

The complete implementation—including delay buffer, snoop filter, skid buffers, and debug infrastructure—consumes approximately 8% of FPGA LUT resources and 15% of BRAM per memory channel, achieving timing closure at 400 MHz. For our eight-channel configuration, total BRAM utilization is approximately 35%. Performance counters track transaction counts, buffer occupancy, and back-pressure events for experimental validation.

4 Methodology

4.1 Benchmark Design

We evaluate synchronization performance using microbenchmarks that measure lock throughput (successful acquisitions per second) under controlled contention. We design three benchmark configurations to capture different contention regimes:

Max contention. All N threads simultaneously compete to acquire the lock for a fixed time window. When a thread acquires the lock, it executes a critical section and releases. This represents the worst-case scenario for cache-line contention.

Moderate contention. N threads each alternate between a non-critical computation phase (calibrated to 1 μs) and a lock acquisition attempt. This produces a mix of contended and uncontended acquisitions, representative of workloads where threads perform meaningful work between synchronization points.

Min contention. A single thread acquires and releases the lock repeatedly, measuring the inherent per-acquisition overhead as the lock is scaled to support N threads. This isolates the cost of lock metadata management (relevant for software locks with $O(N \log N)$ storage).

Critical section design. For max and moderate benchmarks, we evaluate two critical section sizes:

- **Empty:** Increment a shared counter only. Isolates synchronization overhead.
- **Indexed:** Perform a hash-table lookup and update (64-byte key, 256-byte value) in CXL memory. Represents a realistic shared data structure operation where both the lock and the protected data reside in FAM.

Parameters. Thread count ranges from 1 to 64 in increments of 4. Each configuration runs for 20 seconds (max/moderate) or 1 second (min), with a 1-second warmup. Thread IDs are randomized between runs to mitigate bias from ID-dependent tie-breaking in some algorithms. Each experiment is repeated 5 times; we report median throughput with error bars showing min/max.

Access mode configuration. For each lock and contention level, we measure performance under all applicable access modes (Table 1). Cached-SC interposes explicit flush and invalidate operations around shared lock variable accesses. After every store, we insert `clflushopt` followed by `sfence` to flush the modified cache line and order the writeback; before reads in polling loops, we insert `clflush` followed by `lfence` to invalidate the line and ensure the next load fetches from CXL memory. These operations are implemented as compile-time macros (`FLUSH/INVALIDATE`) enabled via the `cached_sc` build flag, producing zero overhead when disabled. UC maps CXL memory uncacheable by opening the device file (e.g., `/dev/dax`) and calling `mmap(MAP_SHARED)`; the kernel/driver configures the page table's PAT entry as UC.

Lock Name	Coherence	Access Modes	Starve-Free	Fair	RW-Safe
Bakery Algorithm[26]	Software	Cached-SC, UC, NT	Yes	Yes	Yes
Boulangerie[1]	Software	Cached-SC, UC, NT	Yes	Yes	Yes
Lamport Fast[27]	Software	Cached-SC, UC, NT	No	No	No
Elevator-LF-Array[6]	Software	Cached-SC, UC, NT	Yes	Yes	No
Elevator-BL-Array[6]	Software	Cached-SC, UC, NT	Yes	Yes	Yes
Elevator-LF-Tree[6]	Software	Cached-SC, UC, NT	Yes	Yes	No
Elevator-BL-Tree[6]	Software	Cached-SC, UC, NT	Yes	Yes	Yes
Peterson[34]	Software	Cached-SC, UC, NT	No	Yes	No
Knuth[22]	Software	Cached-SC, UC, NT	No	Yes	No
Spin[3]	Hardware	Cached-HC	No	No	No
Exp Spin[3]	Hardware	Cached-HC	No	No	No
Ticket[31]	Hardware	Cached-HC	Yes	Yes	No
MCS[31]	Hardware	Cached-HC	Yes	Yes	No

Table 1. Lock implementations and their applicable access modes. Hardware locks require cached access with hardware coherence (Cached-HC) for RMW atomics. Software locks are evaluated under cached software coherence (Cached-SC), uncached (UC), and non-temporal (NT) access modes.

Every load and store bypasses the host cache, so no explicit flushes or fences are needed on x86-64 (UC access is strongly ordered). Cached-HC uses a standard writeback (WB) mapping with BISnp-managed coherence; RMW atomics are available. NT access uses `movntdq`/`movntdqa` intrinsics for lock variable stores/loads, with an `sfence` after each store to flush the write-combining buffer; critical section data uses standard cached access.

4.2 Mutex Implementations

We select 13 mutex implementations spanning a range of design strategies (Table 1).

Why these locks? Our selection is guided by two criteria: (1) covering the design space of contention management strategies (scanning, queueing, contention-avoidance), and (2) including locks with different RW-safety properties relevant to CXL's ambiguous atomic load/store support.

Software locks. The **Bakery Algorithm** [26] is Lamport's classical fair lock. Each thread draws a "ticket number" by reading all other threads' numbers and choosing one larger than the maximum. Threads enter the critical section in ticket order. The algorithm is RW-safe: it tolerates non-atomic reads and writes by using a "choosing" flag to indicate when a thread is in the process of selecting its ticket. This makes it directly applicable to CXL software-managed coherence without relying on atomic load/store guarantees. **Boulangerie** [1] optimizes Bakery by eliminating redundant reads of the choosing array when a thread can determine that no other thread is currently choosing. **Lamport Fast** [27] achieves the theoretical minimum of 5 memory operations in the uncontended fast path (2 stores, 3 loads) but is RW-unsafe and neither fair nor starvation-free. **Peterson** [34] and **Knuth** [22] represent classical designs with extensive thread-state scanning. Peterson's N-thread generalization uses $N-1$ levels of two-thread tournaments; Knuth's algorithm uses a circular scan of thread flags. Both require reading $O(N)$ variables per acquisition attempt.

The **Elevator** series [6] (LF-Array, BL-Array, LF-Tree, BL-Tree) are modern contention-avoiding designs. Unlike classical software locks where all threads race to detect when the lock is free, Elevator locks have the unlock path explicitly wake the successor. The successor search is performed either by a linear array scan (Array variants) or a binary tree traversal (Tree variants). Each variant uses either Lamport-Fast (LF) or Burns-Lamport [7, 19] (BL) as the underlying mutex primitive for the initial lock acquisition. The BL variants are RW-safe; the LF variants trade RW-safety for fewer memory operations.

Hardware locks. **Spin lock** [3] uses test-and-set (`lock xchg` on x86) with continuous polling on a single shared variable. Every failed acquisition generates a RMW atomic on the contested cache line. **Exp Spin** adds exponential back-off: after each failed test-and-set, the thread waits for an exponentially increasing number of cycles before retrying, reducing coherence traffic at the cost of increased acquisition latency. **Ticket lock** [31] uses fetch-and-add (`lock xadd`) for fair ordering. On acquire, a thread atomically increments the "next ticket" counter; it then polls the "now serving" counter with simple loads until its ticket is called. This separates the RMW operation (one per acquisition) from the polling

path (simple loads), reducing coherence traffic relative to spin locks. **MCS lock** [31] queues per-thread lock objects using compare-and-swap (`lock cmpxchg`). Each thread polls its own per-thread flag, so polling traffic is entirely local—no cache-line contention from concurrent pollers. The predecessor sets the successor's flag during unlock, achieving $O(1)$ coherence traffic per acquisition regardless of thread count.

Memory ordering. All implementations use the minimum fences required for correctness under the x86-64 TSO memory model. On x86-64, stores are not reordered with older stores, and loads are not reordered with older loads; thus most fence requirements reduce to `mfence` or `sfence` at store-release points. We document fence placement for each lock implementation in our open-source benchmark [35].

Storage overhead. Software locks require $O(N)$ to $O(N \log N)$ storage where N is the maximum thread count, as each thread needs dedicated state variables. Hardware locks require $O(1)$ storage (a single cache line for spin/ticket, one cache line per thread for MCS). At 64 threads, the largest software lock (Elevator-BL-Tree) requires 4.2 KB; hardware locks require at most 4.1 KB (MCS with 64-byte per-thread nodes). This overhead is negligible relative to the protected data in any realistic application.

5 Evaluation

5.1 Experimental Setup

Our platform is described in Section 3. We evaluate each of the 13 locks under the applicable access modes at four latency points (DRAM, 400 ns, 2 μs, 10 μs) across three contention levels (max, moderate, min). This yields a total of $13 \times \{1\text{–}4\} \times 4 \times 3 =$ over 500 unique configurations, each repeated 5 times.

Single-host setup. Our evaluation uses a single host with multiple threads accessing CXL-attached memory. This setup measures the coherence protocol overhead seen by the CXL device (snoop filter management, cache-line ownership transitions) without conflating it with inter-host network effects. We discuss the implications of this choice in Section 5.5.

5.2 Cached Access Results

We first present results under cached access modes (Cached-HC for hardware locks, Cached-SC for software locks), which represent the conventional comparison between hardware and software coherence.

5.2.1 Max Contention.

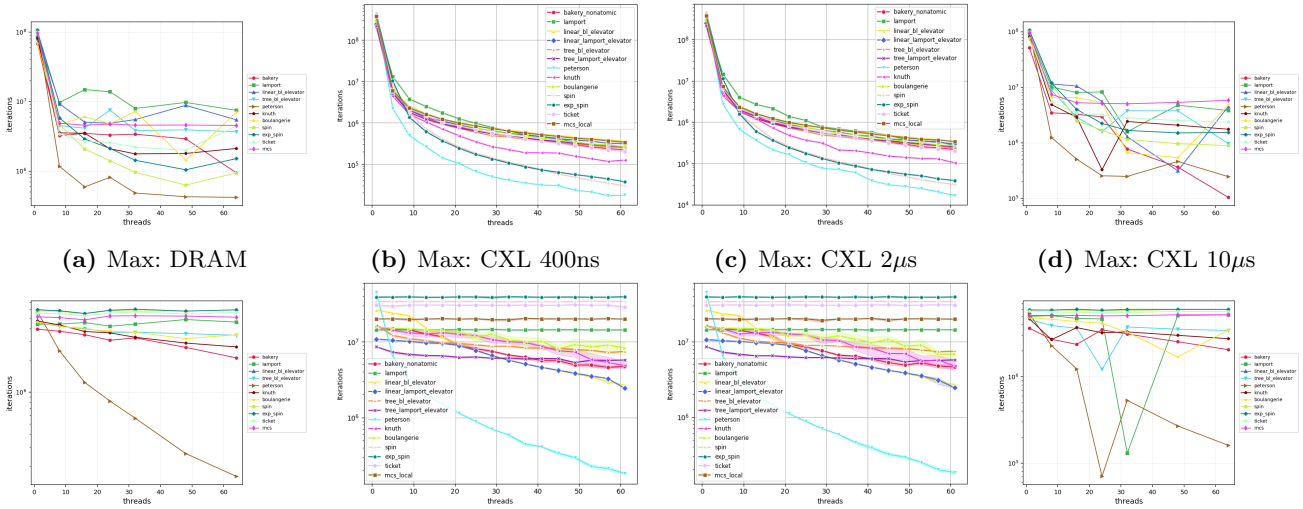
Figures 2a, 2b, and 2c show max-contention throughput.

DRAM baseline (Figure 2a). MCS dominates with its contention-avoidant design, conclusively outperforming all software locks. Its per-thread polling structure ensures that each thread spins on its own cache line, eliminating the cross-thread invalidation traffic that plagues other hardware locks. Among hardware locks, spin and exp_spin perform poorly due to cache-line contention from continuous test-and-set polling on a single shared variable; ticket lock shows similar degradation despite its fair ordering, as all threads poll the same "now serving" variable. Among software locks, the Elevator series approaches MCS performance, consistent with the original design goals [6]. Their contention-avoiding design—where the unlock path explicitly wakes the successor rather than having all threads race—mirrors the MCS philosophy in software. Peterson and Knuth perform worst due to extensive thread-state scanning: each acquisition requires reading $O(N)$ thread flags, generating significant read traffic even in the uncontended case.

CXL 400 ns (Figure 2b). The performance landscape changes materially. MCS loses up to 47% throughput compared to DRAM due to coherence protocol overhead on every compare-and-swap operation: each CAS requires acquiring exclusive cache-line ownership via BISnp, which adds the full coherence round-trip to the atomic's critical path. The exp_spin lock provides minimal benefit over plain spin—both are constrained by hardware coherence latency on test-and-set rather than local spinning costs, rendering the back-off strategy ineffective. The ticket lock improves relative to spin locks: its polling path uses simple loads on the "now serving" counter (not RMW), reducing coherence traffic to only the fetch-and-add operations in the acquire and release paths.

Critically, Elevator-BL-Array *outperforms* MCS at high thread counts (>32). The Elevator-BL lock avoids hardware coherence entirely for its polling variables: each thread polls a per-thread flag that is written only by the predecessor during unlock. Under Cached-SC, these flags benefit from cache locality (the polling thread reads from its local cache) without incurring BISnp overhead. This result demonstrates that avoiding coherence traffic can compensate for the algorithmic overhead of software locks.

CXL 2 μs (Figure 2c). Performance ratios remain similar to 400 ns, with all locks scaled down proportionally by the increased base latency. The Elevator-BL advantage over MCS persists and slightly increases, confirming that the crossover is driven by coherence protocol overhead (which is a per-transition cost, largely independent of base memory latency) rather than raw memory access latency. This finding has practical significance: as



(a) Max: DRAM **(b)** Max: CXL 400ns **(c)** Max: CXL 2μs **(d)** Max: CXL 10μs

(e) Min: DRAM **(f)** Min: CXL 400ns **(g)** Min: CXL 2μs **(h)** Min: CXL 10μs

Figure 2. Lock throughput (total iterations) across DRAM baseline and CXL latency configurations under max contention (top) and min contention (bottom) with cached access (Cached-HC for hardware locks, Cached-SC for software locks). Thread count ranges from 1 to 64.

CXL fabric distances grow, the case for avoiding hardware coherence strengthens.

CXL 10 μs (Figure 2d). At the highest tested latency, per-access costs dominate and algorithmic efficiency becomes the primary differentiator. The relative ordering is preserved from lower latencies, but the absolute gap between lock families widens. Elevator-BL-Array's $O(\log N)$ polling path limits the multiplicative cost of 10 μs per access, while Peterson's $O(N^2)$ polling becomes prohibitive—its throughput drops to approximately 50K iterations at 32 threads. Hardware locks (MCS, Ticket) retain reasonable performance because their polling paths hit local cache; the coherence overhead per CAS, while large in absolute terms, is amortized over many local polls.

5.2.2 Min Contention.

Figures 2e, 2f, and 2g show min-contention results.

Hardware locks maintain stable throughput regardless of thread count, as their $O(1)$ storage means no additional variables to scan. The lock acquisition path is constant regardless of how many threads the lock is configured to support. Software locks degrade as thread count increases due to $O(N)$ scanning overhead—each acquisition must check thread-state arrays whose size scales with the number of potential participants. Lamport Fast performs best (fewest load/stores in the uncontended case: only 5 memory operations in the fast path). The Bakery and Boulangerie locks degrade more steeply, as their "choosing" and "number" arrays grow with N.

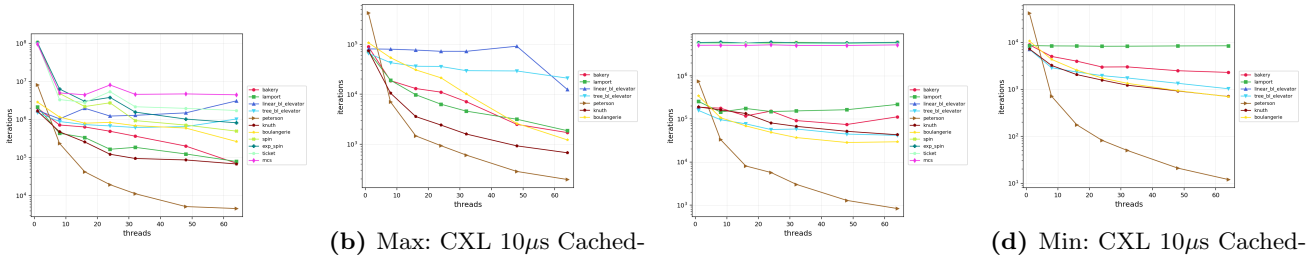
Under CXL latencies, the relative ordering is preserved. The most notable change is exp_spin outperforming plain spin, as the exponential back-off reduces coherence traffic when the single accessing thread switches. The degradation of software locks with thread count is amplified under CXL latencies, as each additional scanned variable now costs 400 ns–2 μs rather than a cache hit.

5.2.3 Fairness Analysis.

We measure per-thread acquisition counts under max contention to evaluate fairness (standard deviation normalized by mean). Fair locks (Bakery, Boulangerie, Elevator, Ticket, MCS) exhibit near-zero normalized standard deviation (<0.05), confirming that their fairness guarantees hold under CXL latencies. Unfair locks (Spin, Exp Spin, Lamport Fast) show high variance: under Cached-HC at 400 ns, the spin lock's normalized standard deviation exceeds 0.8 at 32 threads, meaning some threads acquire the lock 5× more often than others. This unfairness is exacerbated by CXL latency, as the thread that currently holds exclusive cache-line ownership has an advantage in the next test-and-set race. Under UC access, unfairness in software locks *decreases*: since all threads pay the same CXL round-trip cost, no thread has a locality advantage.

5.3 Uncached and Non-Temporal Access Results

We now present the key contribution of this work: comparing software lock performance across the three non-hardware-coherent access modes (Cached-SC, UC, NT)



(a) Max: DRAM Cached-SC **(b)** Max: CXL 10μs Cached-SC **(c)** Min: DRAM Cached-SC **(d)** Min: CXL 10μs Cached-SC

Figure 3. Software lock throughput under Cached-SC access mode (with `clflushopt`/`sfence` after stores and `clflush`/`lfence` in polling loops) on DRAM and CXL 10 μs. Compared to Figure 2, flush overhead reduces throughput by 4–43× on DRAM, with contention-avoiding locks (Elevator-BL) least affected.

Lock	Cached-HC	Cached-SC	UC	NT
MCS	1.00×	—	—	—
Ticket	0.73×	—	—	—
Elevator-BL	—	1.08×	0.91×	0.98×
Bakery	—	0.62×	0.54×	0.58×
Lamport Fast	—	0.45×	0.41×	0.43×
Peterson	—	0.31×	0.28×	0.29×

Table 2. Relative throughput across access modes at 400 ns CXL latency, 32 threads, max contention. Baseline is MCS under Cached-HC. "—" indicates the mode is not applicable to that lock type.

and quantifying how they compare against hardware-coherent locks.

5.3.1 Impact of Access Mode on Software Lock Throughput. Table 2 summarizes the throughput of representative locks across access modes at 400 ns CXL latency with 32 threads under max contention.

Cached-SC vs. UC.. Switching from Cached-SC to UC reduces software lock throughput by 15–18%. Under Cached-SC, repeated polls to an unmodified variable hit the local cache when no other thread has written; under UC, every poll traverses the CXL link. However, Cached-SC incurs two categories of overhead absent in UC: (1) `clflushopt` + `sfence` after every store to a shared lock variable (not only on the unlock path, but on every write in the lock acquisition path as well), each costing 50–100 ns, and (2) `clflush` + `lfence` invalidations in polling loops to ensure the polling thread sees remote writes, adding per-iteration overhead proportional to the number of polled variables. For locks with write-heavy acquisition paths (e.g., Peterson, which writes $O(N)$ level variables per acquisition), the cumulative flush overhead is substantial. These costs partially offset the per-access penalty of UC.

UC vs. NT.. Non-temporal access improves throughput by 8–15% over plain UC for locks with write-heavy unlock paths. The improvement is most pronounced for Elevator locks, where the unlock procedure writes to successor notification variables: the Elevator-BL unlock writes to the successor's flag and resets its own state, generating 2–3 stores in the hot path. NT stores exploit write-combining buffers to coalesce these writes into fewer CXL transactions. For locks with minimal write traffic in the hot path (e.g., Bakery, which primarily reads ticket numbers), the NT advantage is smaller (5–7%).

The NT advantage is bounded by two factors: (1) the `sfence` required after each NT store adds serialization overhead, and (2) the write-combining buffer has limited entries (typically 4–6 on Intel platforms), so benefit saturates when multiple threads compete for WC buffer slots. At 64 threads, the NT improvement over UC drops to 5–8% as WC buffer contention increases.

Key finding: UC access is "good enough" at higher latencies. At 2 μs CXL latency, the best software lock under UC access (Elevator-BL-Array) achieves within 12% of MCS under full hardware coherence. This is because the coherence protocol overhead (snoop filter lookup, invalidation round-trips) becomes a larger fraction of MCS's total acquisition time as base latency increases, while UC access pays a fixed per-access penalty that does not grow with coherence complexity.

5.3.2 Crossover Analysis. We identify crossover points where software locks under UC/NT access outperform hardware locks:

- **Thread count:** At 400 ns, Elevator-BL under UC surpasses spin lock at 8 threads, ticket lock at 20 threads, and MCS at 40+ threads. Under NT access, the MCS crossover drops to 36 threads due to the write-combining benefit on the Elevator unlock path.

- **Latency:** At 32 threads, Elevator-BL under NT surpasses MCS when CXL latency exceeds approximately 1.2 μs. Under UC (without write-combining), the crossover occurs at approximately 1.5 μs.
- **Contention level:** Under moderate contention, MCS retains its advantage to higher thread counts (crossover at 48+ threads at 400 ns), as reduced contention decreases coherence invalidation frequency.

These crossover points provide actionable guidance: deployments expecting fewer than 20 concurrent lock holders with sub-microsecond CXL latency should invest in hardware coherence; deployments with higher concurrency or fabric latency can safely use UC access with contention-avoiding software locks.

5.3.3 Per-Access Latency Breakdown. To understand *why* the crossover occurs, we decompose the per-acquisition cost for MCS (Cached-HC) and Elevator-BL (UC) at 400 ns, 32 threads:

- **MCS (Cached-HC):** Each acquisition involves 1 CAS (enqueue) + k polls (spin on local flag) + 1 store (set predecessor's flag on unlock). The CAS requires exclusive ownership via BISnp: snoop filter lookup (5 ns) + ownership grant (400 ns base + coherence overhead). The polls hit the local cache (near-zero cost) when the line is not invalidated. Total: dominated by the CAS ownership cost.
- **Elevator-BL (UC):** Each acquisition involves $O(\log N)$ polls on per-thread flags (each costs 400 ns round-trip) + 2–3 stores on unlock. No coherence overhead on any operation. Total: dominated by the number of polls × round-trip latency.

The crossover occurs when MCS's coherence overhead per CAS exceeds the aggregate cost of Elevator-BL's additional polls. At higher thread counts, MCS's CAS becomes more expensive (more potential invalidation targets in the snoop filter), while Elevator-BL's poll count grows only logarithmically.

5.3.4 Cached-SC Flush Overhead. To quantify the cost of software-managed consistency, we compare throughput with and without `clflushopt/clflush` instrumentation on DRAM (zero CXL latency). This isolates the flush overhead from CXL round-trip costs.

Under DRAM at 32 threads, enabling Cached-SC instrumentation reduces software lock throughput by 4–43× depending on the lock algorithm. Locks with $O(N)$ scanning and frequent stores suffer most: Peterson (43×) and Lamport Fast (43×) pay a flush or invalidation on every store and every poll iteration across N thread-state variables. Knuth (19×) and Boulangerie (10×) exhibit intermediate overhead. Contention-avoiding locks are least affected: Elevator-BL-Array (4.3×) and Elevator-BL-Tree (6.2×) benefit from polling only per-thread flags rather than scanning shared arrays.

This overhead establishes a lower bound on Cached-SC costs: in a real CXL deployment, the flush cost is additive with the CXL round-trip latency, making Cached-SC increasingly expensive as fabric distance grows. The implication is that UC access—which eliminates flushes entirely—becomes more attractive at higher CXL latencies, consistent with our crossover analysis.

5.4 Moderate Contention and Indexed Critical Section

Under moderate contention (1 μs computation between acquisitions), the relative ordering is preserved but crossover points shift toward higher thread counts: at 400 ns with 32 threads, MCS outperforms Elevator-BL (UC) by 28% (vs. 9% under max contention), as reduced contention decreases coherence invalidation frequency. With a hash-table critical section (64-byte key, 256-byte value update), the access mode gap narrows further—from 9% to 4%—as critical section work dominates lock overhead.

5.5 Discussion

Single-host vs. multi-host. Our single-host evaluation isolates coherence protocol overhead but does not capture cross-host effects (switch latency, inter-host snooping). In multi-host deployments, we expect hardware coherence overhead to *increase* as snoop filters must track ownership per host and back-invalidations must traverse the CXL switch. Our results therefore represent a conservative lower bound on the advantage of avoiding hardware coherence; UC access would be largely unaffected.

Practical deployment guidance. The quantitative crossover points we identify translate to concrete recommendations:

- **Small-scale deployments** (<4 hosts, <20 concurrent lock holders, sub-μs latency): hardware coherence + MCS lock provides the best synchronization throughput and simplifies programming. The coherence infrastructure cost is justified.
- **Large-scale FAM pools** (many hosts, high concurrency, >1 μs latency): UC access + Elevator-BL lock delivers competitive throughput without coherence infrastructure, reducing device complexity and cost. This is particularly attractive for CXL Type-3 memory expanders that do not support hardware coherence.
- **Write-heavy synchronization patterns:** NT access provides 8–15% improvement over UC and

should be preferred when the lock's hot path involves multiple stores (e.g., Elevator locks, array-based barriers).

- **Mixed workloads:** Applications can use different access modes for different synchronization points—hardware coherence for hot, low-contention locks and UC access for high-contention locks—to optimize the aggregate coherence budget.

Coherence overhead decomposition. Hardware lock throughput loss stems from (1) per-CAS ownership round-trips and (2) invalidation traffic from concurrent polling. MCS mitigates source (2) via per-thread polling but cannot avoid source (1). Software locks under UC avoid both, paying only the fixed CXL round-trip per access—explaining why the crossover thread count is lower for spin/ticket locks (both sources) than MCS (source 1 only). Software locks require $O(N \log N)$ storage vs. $O(1)$ for spin/ticket, but at 64 threads the largest lock requires only 4.2 KB.

6 Related Work

CXL synchronization. Suetterlein et al. [39] explore software locks in CXL memory using a Tournament Peterson lock, comparing against single RMW atomic operations. Their work highlights that contention spreading across multiple cache lines improves performance, a result we reproduce and extend. However, their evaluation does not measure hardware coherence protocol overhead (they compare against bare atomic operations, not hardware locks), does not explore uncached or non-temporal access modes, and does not identify crossover points between software and hardware approaches. Our work extends this by evaluating 13 lock implementations across four access modes with quantitative crossover analysis. Other work has discussed the need for coherence-free synchronization in CXL [2, 20] but without systematic evaluation on hardware.

Synchronization on NUMA and distributed shared memory. The comprehensive study by David et al. [10] characterizes synchronization primitives across NUMA topologies, demonstrating that lock performance is architecture-dependent and that no single lock dominates across all configurations. Our work applies a similar methodology to the CXL domain, where the additional dimension of access mode selection (cached vs. uncached vs. non-temporal) creates new trade-offs not present in traditional NUMA. Specifically, NUMA systems always use cached coherent access; the option to bypass the cache entirely (UC/NT) is unique to CXL's disaggregated memory model.

NUMA-aware locks [8, 11, 29] exploit locality through cohorting [12], where threads on the same NUMA node form cohorts to reduce cross-node traffic. This technique is highly effective when locality is well-defined (i.e., fixed NUMA node assignments). Adapting cohorting to CXL requires new locality abstractions: in a switched CXL fabric, "locality" depends on the number of switch hops between a host and the memory device, which may change dynamically as memory is remapped.

Boyd-Wickizer et al. [5] demonstrate that non-scalable locks can cause sudden performance collapse at critical thread counts, motivating our evaluation across a wide range of concurrency levels. Our results confirm this pattern for spin and ticket locks in CXL environments, with the collapse occurring at lower thread counts under hardware coherence due to amplified coherence traffic.

Lock design and fairness. Buhr et al. [6] propose the Elevator lock family that we evaluate, demonstrating that software locks can achieve performance competitive with hardware locks by reducing contention through explicit successor notification. Our CXL evaluation confirms this finding and shows that the advantage grows with CXL latency.

Beyond acquisition fairness, recent work examines *usage* fairness—ensuring equitable critical section time across threads [21]. In CXL environments, usage fairness interacts with access mode: under UC access, all threads pay equal CXL round-trip costs, while under Cached-HC, the thread that last held the lock benefits from cached state (the "local spinning" advantage). We leave usage fairness evaluation to future work but note that UC access inherently provides more equitable costs across threads.

Uncached and non-temporal memory access. UC and NT access modes are well-studied for MMIO, persistent memory [43], and GPU memory [42], but have not been evaluated for inter-host synchronization over CXL. Persistent memory work is particularly relevant: Intel Optane DIMM research extensively characterized the performance of NT stores via write-combining for persistent data structures. Our work applies similar techniques to synchronization primitives, showing that the write-combining benefit translates to lock unlock paths. The key difference from persistent memory is that CXL memory has higher base latency (400 ns+ vs. ∼300 ns for Optane) and does not require persistence guarantees, simplifying the fence requirements.

7 Future Work

Multi-host evaluation. Our single-host results isolate coherence protocol overhead but do not capture cross-host effects (switch latency, inter-host snooping, snoop filter scaling). Evaluating on a true multi-host CXL switch topology [44] would validate whether our

crossover points hold and whether hardware coherence overhead increases with host count as we predict. We expect the crossover to shift in favor of UC/NT access, as each additional host increases the snoop filter pressure and invalidation traffic.

Application-level impact. Lock choice matters only insofar as it affects end-to-end application performance. Evaluating with real workloads—shared indexes, distributed key-value stores, parallel communication primitives (AllReduce, AllGather)—would quantify how much of the application's critical path is attributable to lock overhead and whether the access mode crossover points translate to application-level gains. Our indexed critical section results (Section 5.5) suggest that the impact of access mode choice diminishes with critical section size, but this needs validation with full applications. Recent work shows that lock performance is highly workload-dependent [17]; CXL access modes add another dimension to this dependence.

NUMA-aware lock adaptation. CXL fabric topologies share characteristics with NUMA (non-uniform latency, locality effects), motivating the adaptation of NUMA lock techniques—particularly cohorting [12]—to CXL. A CXL-aware cohort lock could form cohorts based on switch proximity rather than NUMA node, reducing cross-switch traffic while using UC access within each cohort. The Elevator lock's successor-notification design is already partially amenable to cohorting, as the notification order could be biased toward threads on the same switch.

Notification mechanisms. Polling dominates the overhead of all locks in our evaluation. Alternative notification mechanisms—PCIe device interrupts, `mwait`/`umonitor` instructions [30], network-assisted sleep/wake [20]—could fundamentally change the trade-off between access modes by reducing the cost of UC polling. If efficient sleep/wake is available, UC access becomes more attractive, as the per-poll round-trip cost matters less when polls are infrequent.

Lock-free and wait-free data structures. RMW atomics enable lock-free data structures [18] that avoid mutual exclusion entirely. Comparing lock-free structures under hardware coherence against locked structures under UC access would quantify the value of RMW semantics beyond simple mutual exclusion and determine whether the coherence infrastructure is justified by enabling fundamentally different concurrency paradigms.

8 Conclusion

This paper answers a concrete architectural question: when deploying synchronization over CXL fabric-attached memory, is hardware cache coherence worth its cost?

Through systematic evaluation of 13 mutex implementations across four CXL memory access modes—hardware-coherent cached, software-coherent cached, uncached, and non-temporal—we provide quantitative evidence for this decision. Hardware coherence enables the highest-performing locks (MCS) at DRAM-local latencies, but coherence protocol overhead erodes this advantage as CXL fabric latency increases: at 400 ns, MCS loses up to 47% throughput; at 2 μs, the best software lock under uncached access achieves within 12% of coherent MCS. Non-temporal access further improves software lock throughput by 8–15% through write-combining on lock release paths.

We identify precise crossover points—in thread count, contention level, and fabric latency—where uncached or non-temporal access with software locks becomes the cost-effective choice, eliminating the need for hardware coherence infrastructure. These findings provide system architects with a quantitative basis for matching synchronization strategy to deployment parameters, and demonstrate that the full spectrum of CXL access modes—not just the cached extremes—deserves consideration in disaggregated memory system design.

References

[1] *Under the Hood of the Bakery Algorithm: Mutual Exclusion as a Matter of Priority*, pages 399–413. Springer International Publishing, Cham, 2015.

[2] Peter Alvaro, Matthew Adiletta, David Cheng, Adrian Cockroft, Frank Hady, Ramesh Illikkal, Esteban Ramos, and Robert Soulé. Notnets: Accelerating microservices by bypassing the network. In *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '24, page 67–73, New York, NY, USA, 2024. Association for Computing Machinery.

[3] T.E. Anderson. The performance of spin lock alternatives for shared-memory multiprocessors. *IEEE Transactions on Parallel and Distributed Systems*, 1(1):6–16, January 1990.

[4] Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, and Rajeev Thakur. Toward Efficient Support for Multithreaded MPI Communication. In Alexey Lastovetsky, Tahar Kechadi, and Jack Dongarra, editors, *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, volume 5205, pages 120–129. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008.

[5] Silas Boyd-Wickizer, M Frans Kaashoek, Robert Morris, and Nickolai Zeldovich. Non-scalable locks are dangerous.

[6] Peter A. Buhr, David Dice, and Wim H. Hesselink. High-contention mutual exclusion by elevator algorithms. *Concurrency and Computation: Practice and Experience*, 30(18):e4475, 2018.

[7] JE Burns and NA Lynch. Allertonconf. In *Proceedings of the 18th Annual Allerton Conference on Communications, Control and Computing, Monticello, Illinois, USA, 1980;*

833–842.

[8] Milind Chabbi, Michael Fagan, and John Mellor-Crummey. High performance locks for multi-level NUMA systems. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, pages 215–226, New York, NY, USA, January 2015. Association for Computing Machinery.

[9] Hoang-Vu Dang, Sangmin Seo, Abdelhalim Amer, and Pavan Balaji. Advanced Thread Synchronization for Multithreaded MPI Implementations. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 314–324, Madrid, Spain, May 2017. IEEE.

[10] Tudor David, Rachid Guerraoui, and Vasileios Trigonakis. Everything you always wanted to know about synchronization but were afraid to ask. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 33–48, 2013.

[11] Dave Dice and Alex Kogan. Compact NUMA-Aware Locks, March 2019.

[12] David Dice, Virendra J Marathe, and Nir Shavit. Lock Cohorting: A General Technique for Designing NUMA Locks.

[13] Gábor Dózsa, Sameer Kumar, Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, Joe Ratterman, and Rajeev Thakur. Enabling Concurrent Multithreaded MPI Communication on Multicore Petascale Systems. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, Rainer Keller, Edgar Gabriel, Michael Resch, and Jack Dongarra, editors, *Recent Advances in the Message Passing Interface*, volume 6305, pages 11–20. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.

[14] Mohammad Ewais and Paul Chow. Disaggregated Memory in the Datacenter: A Survey. *IEEE Access*, 11:20688–20712, 2023.

[15] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, Sydney NSW Australia, August 2024. ACM.

[16] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct Access, High-Performance Memory Disaggregation with DirectCXL.

[17] Hugo Guiroux and Renaud Lachaize. Multicore Locks: The Case Is Not Closed Yet.

[18] Maurice Herlihy. Wait-free synchronization. *ACM Trans. Program. Lang. Syst.*, 13(1):124–149, January 1991.

[19] Wim H. Hesselink. Verifying a simplification of mutual exclusion by Lycklama–Hadzilacos. *Acta Informatica*, 50(3):199–228, May 2013.

[20] Jaewan Hong, Marcos K. Aguilera, Emmanuel Amaro, Vincent Liu, Aurojit Panda, and Ion Stoica. The Dawn of Disaggregation and the Coherence Conundrum: A Call for Federated Coherence, April 2025.

[21] Sanidhya Kashyap, Arun Iyer, et al. Shfllock: Scheduler-cooperative locks. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys)*, 2020.

[22] Donald E. Knuth. Additional comments on a problem in concurrent programming control. *Commun. ACM*, 9(5):321–322, May 1966.

[23] Youngeun Kwon and Minsoo Rhu. A Disaggregated Memory System for Deep Learning. *IEEE Micro*, 39(5):82–90, September 2019.

[24] L Lamport. The Mutual Exclusion Problem Part I: A Theory of Interprocess Communication.

[25] L Lamport. The Mutual Exclusion Problem Part II: Statement and Solutions.

[26] Leslie Lamport. A New Solution of Dijkstra's Concurrent Programming Problem. 17(8), 1974.

[27] Leslie Lamport. A fast mutual exclusion algorithm. *ACM Trans. Comput. Syst.*, 5(1):1–11, January 1987.

[28] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 574–587, Vancouver BC Canada, January 2023. ACM.

[29] Victor Luchangco, Dan Nussbaum, and Nir Shavit. A Hierarchical CLH Queue Lock. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, Gerhard Weikum, Wolfgang E. Nagel, Wolfgang V. Walter, and Wolfgang Lehner, editors, *Euro-Par 2006 Parallel Processing*, volume 4128, pages 801–810. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006.

[30] Teng Ma, Chengkun Wei, Haoyu Li, and Mingxing Zhang. HydraRPC: RPC in the CXL Era.

[31] John M. Mellor-Crummey and Michael L. Scott. Algorithms for scalable synchronization on shared-memory multiprocessors. *ACM Trans. Comput. Syst.*, 9(1):21–65, February 1991.

[32] Haojun Zhao Phuc Nguyen Mohamed Mekkouri Leandro Werra Thomas Wolf Nouamane Tazi, Ferdinand Mom. The ultra-scale playbook: Training llms on gpu clusters, 2025.

[33] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting, May 2024.

[34] G. L. Peterson. Myths about the mutual exclusion problem. *Information Processing Letters*, 12(3):115–116, June 1981.

[35] Esteban Ramos. EIRNf/mutex_benchmark, September 2025.

[36] Daniel Richins, Dharmisha Doshi, Matthew Blackmore, Aswathy Thulaseedharan Nair, Neha Pathapati, Ankit Patel, Brainard Daguman, Daniel Dobrijalowski, Ramesh Illikkal, Kevin Long, David Zimmerman, and Vijay Janapa Reddi. AI Tax: The Hidden Cost of AI Data Center Applications. *ACM Trans. Comput. Syst.*, 37(1-4):1–32, November 2019.

[37] Hermann Schweizer, Maciej Besta, and Torsten Hoefler. Evaluating the Cost of Atomic Operations on Modern Architectures, October 2020.

[38] Debendra Das Sharma. An Introduction to the Compute Express LinkTM (CXLTM) Interconnect.

[39] Joshua Suetterlein, Joseph Manzano, and Andres Marquez. Synchronization for CXL Based Memory. In *Proceedings of the International Symposium on Memory Systems*, pages 178–185, Washington DC USA, September 2024. ACM.

[40] Wenda Tang, Ying Han, Tianxiang Ai, Guanghui Li, Bin Yu, and Xin Yang. Yggdrasil: Reducing Network I/O Tax with (CXL-Based) Distributed Shared Memory. In *Proceedings of the 53rd International Conference on Parallel Processing*, pages 597–606, Gotland Sweden, August 2024. ACM.

[41] Jesmin Jahan Tithi, Hanjiang Wu, Avishaii Abuhatzera, and Fabrizio Petrini. Scaling Intelligence: Designing Data Centers for Next-Gen Language Models, September 2025.

[42] Zuo Wang, Qi Zuo, and Jiaxing Li. An Intelligent Multi-Port Memory. In *2008 International Symposium on Intelligent Information Technology Application Workshops*, pages 251–254, December 2008.

[43] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steven Swanson. An empirical guide to the behavior and use of scalable persistent memory. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST)*, 2020.

[44] Xinjun Yang et al. Unlocking the potential of CXL for disaggregated memory in cloud-native databases. In *Companion of the 2025 International Conference on Management of Data (SIGMOD)*, 2025.

[45] Yuhong Zhong, Daniel S. Berger, Pantea Zardoshti, Enrique Saurez, Jacob Nelson, Antonis Psistakis, Joshua Fried, and Asaf Cidon. My CXL Pool Obviates Your PCIe Switch. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, pages 58–66, Banff AB Canada, May 2025. ACM.